Microservices Design Principles

A Complete In-Depth Guide with Real-World Examples

Single Responsibility • Loose Coupling • High Cohesion

Domain-Driven Design • API Design • Communication Patterns

Resilience • Data Management • Observability • Security

Deployment • Team Topology • Anti-Patterns

15 Core Principles • Good vs Bad Examples • Code Patterns • Checklists

Table of Contents

CHAPTER 1

Microservices Foundations

What are Microservices, Why They Matter & The 15 Design Principles

1.1 What Are Microservices?

Microservices Architecture

Microservices is an architectural style where an application is composed of small, independently deployable services, each responsible for a specific business capability. Each service is built around a business domain, communicates over network APIs, has its own data store, and is deployed and scaled independently. The style emerged from the practical problems of large monolithic applications at companies like Netflix, Amazon, and Uber.

Aspect	Monolith	Microservices
Deployment	Deploy entire app as one unit	Deploy each service independently
Scaling	Scale whole application	Scale individual services by demand
Technology	Single language/framework	Polyglot – each service chooses its own
Database	One shared database	Each service owns its own database
Failure	One bug can crash everything	Failures isolated to one service
Team Ownership	Large teams, shared codebase	Small autonomous teams own services
Complexity	Simple to start; complex at scale	Complex from day one; manageable at scale
Testing	Easier end-to-end testing	Integration testing is harder
Latency	In-process calls, fast	Network calls, added latency
Best For	Small apps, single teams, MVPs	Large apps, many teams, high scale

1.2 The 15 Core Design Principles – Overview

#	Principle	Category	Key Question
P1	Single Responsibility	Service Design	Does each service do ONE thing well?
P2	Bounded Context (DDD)	Service Design	Are service boundaries aligned to business domains?
P3	Loose Coupling	Service Design	Can services change without affecting each other?
P4	High Cohesion	Service Design	Do all parts of a service belong together?
P5	API-First Design	Service Design	Is the contract defined before implementation?

#	Principle	Category	Key Question
P6	Smart Endpoints, Dumb Pipes	Communication	Does logic live in services, not infrastructure?
P7	Sync vs Async Communication	Communication	Is the right communication style chosen?
P8	Event-Driven Architecture	Communication	Are services communicating via events?
P9	Consumer-Driven Contracts	Communication	Are API changes tested against consumers?
P10	Design for Failure	Resilience	Does the system handle partial failures gracefully?
P11	Bulkhead & Isolation	Resilience	Are failures isolated to prevent cascades?
P12	Circuit Breaker & Fallback	Resilience	Are failing services short-circuited quickly?
P13	Database Per Service	Data	Does each service own its own data store?
P14	Eventual Consistency & Saga	Data	Are distributed transactions handled via Saga?
P15	CQRS & Event Sourcing	Data	Are reads and writes separated for scalability?

CHAPTER 2

Service Design Principles

Single Responsibility, Bounded Context, Coupling, Cohesion & API-First

P1 Single Responsibility Principle

Each service does one thing and does it well

Single Responsibility

A microservice should be responsible for one and only one business capability. It should have one reason to change. When a service tries to do too much, it becomes a 'mini-monolith' — hard to understand, test, deploy, and scale independently. Ask: 'If this service were to grow, would ALL that growth belong here?'

+ Good Practice

```
# GOOD: OrderService handles only order lifecycle
OrderService:
- createOrder(customerId, items)
- confirmOrder(orderId)
- cancelOrder(orderId)
- getOrderStatus(orderId)
- listOrdersByCustomer(customerId)

PaymentService:
- processPayment(orderId, amount)
- refundPayment(paymentId)

NotificationService:
- sendOrderConfirmationEmail()
- sendShippingNotification()
```

- Anti-Pattern

```
# BAD: OrderService does too much
OrderService:
- createOrder() <- order domain
- processPayment() <- payment domain
- sendEmail() <- notification domain
- updateInventory() <- inventory domain
- generateInvoicePDF() <- billing domain
- applyLoyaltyPoints() <- loyalty domain

# This is a distributed monolith masquerading
# as a microservice — avoid this!
```

P2 Bounded Context & Domain-Driven Design

Service boundaries align to business domain boundaries

Bounded Context (DDD)

A Bounded Context defines the boundary within which a particular domain model applies. In DDD (Domain-Driven Design), each microservice should correspond to one Bounded Context. The same word can mean different things in different contexts (e.g., 'Customer' means different things in Sales vs Support vs Billing), and each context should define its own model without sharing or polluting other contexts.

- **Ubiquitous Language** – Each service team develops a common language aligned to the business domain.

- **Context Map** – Documents relationships between bounded contexts (upstream/downstream, shared kernel).
- **Aggregates** – Cluster of domain objects treated as a unit for data changes; defines transaction boundary.
- **Domain Events** – Something that happened in the domain (OrderPlaced, PaymentProcessed) — facts.

Bounded context – each service owns its own model

```
// GOOD: Each service has its own model of "Customer"

// OrderService context: Customer is just an ID and shipping address
public record OrderCustomer(
String customerId,
String shippingAddress,
String email) {}

// LoyaltyService context: Customer is about points and rewards
public record LoyaltyMember(
String customerId,
int points,
String tier, // BRONZE, SILVER, GOLD
LocalDate memberSince) {}

// SupportService context: Customer is about tickets and history
public record SupportCustomer(
String customerId,
List openTickets,
String preferredChannel) {}

// NO shared Customer class across services!
// Each context owns its Customer model
```

P3 Loose Coupling

Services can change independently without breaking each other

Loose Coupling

Loose coupling means that a change in one service does not require a change in another. Highly coupled services deploy together, fail together, and cannot be developed independently. Coupling can be temporal (synchronous call means both must be running), behavioural (shared logic), or data (shared database schema).

Coupling Type	Description	Solution
Temporal	Service A must be available for B to work (sync call)	Use async messaging / queues
Data	Services share the same database table or schema	Database per service; event sync
Behavioural	Services share business logic (shared libraries with domain logic)	Keep shared libs for utilities only

Coupling Type	Description	Solution
Deployment	Services must be deployed at the same time	Independent deployability; API versioning
Contract	Consumer hardcodes producer's internal data structure	Consumer-driven contracts; stable API

+ Good Practice

```
// GOOD: Async messaging – loose temporal coupling
@Service
public class OrderService {
private final KafkaTemplate kafka;

public Order placeOrder(OrderRequest req) {
Order order = orderRepo.save(new Order(req));
// Fire event – don't wait for response
kafka.send('order-events',
new OrderPlacedEvent(order));
return order;
// PaymentService handles event independently
}
}
```

- Anti-Pattern

```
// BAD: Synchronous chain – tight temporal coupling
@Service
public class OrderService {
private final PaymentClient payment;
private final InventoryClient inventory;
private final NotifyClient notify;

public Order placeOrder(OrderRequest req) {
payment.charge(req); // fails? order fails
inventory.reserve(req); // fails? partial state
notify.sendEmail(req); // fails? order stuck
return orderRepo.save();
}
}
```

P4 High Cohesion

Everything inside a service belongs together and changes together

High Cohesion

Cohesion measures how closely related the responsibilities of a service are. A highly cohesive service has all its functionality centred around one theme — changes to one part often require changes to others within the same service. A service with low cohesion has unrelated functionality bundled together, making it harder to reason about and change.

- **Functional Cohesion** (best) – All parts contribute to a single, well-defined task.
- **Sequential Cohesion** – Output of one part is input to another (process pipeline).
- **Communicational Cohesion** – Parts operate on the same data or resource.
- **Coincidental Cohesion** (worst) – Parts are grouped arbitrarily with no logical relationship.

i The 'Goldilocks' principle: services should be not too big (distributed monolith) and not too small (nanoservices with too much overhead). Size should be determined by business capability, not lines of code.

P5 API-First Design

Define the contract before writing any implementation code

API-First Design

In API-First design, the API contract (OpenAPI spec, Protobuf, AsyncAPI) is written and agreed upon BEFORE any implementation begins. This enables parallel development (frontend and backend teams work simultaneously), enforces a stable contract, enables mock servers, auto-generates documentation, and makes breaking changes visible.

API-First with OpenAPI

```
# GOOD: Define OpenAPI spec first, then implement
# openapi: 3.0.3
paths:
/api/orders:
post:
summary: Place a new order
requestBody:
required: true
content:
application/json:
schema:
$ref: "#/components/schemas/CreateOrderRequest"
responses:
"201":
description: Order created
content:
application/json:
schema:
$ref: "#/components/schemas/OrderResponse"
"400":
description: Validation error
content:
application/json:
schema:
$ref: "#/components/schemas/ErrorResponse"

# Generate server stubs from spec:
# openapi-generator-cli generate -i openapi.yaml -g spring -o ./server
# Generate client SDKs for consumers:
# openapi-generator-cli generate -i openapi.yaml -g java -o ./client
```

CHAPTER 3

Communication Principles

Smart Endpoints, Sync vs Async, Event-Driven & Consumer Contracts

P6 Smart Endpoints, Dumb Pipes

Business logic lives in services, not in the messaging infrastructure

Smart Endpoints, Dumb Pipes

This principle rejects the ESB (Enterprise Service Bus) pattern where the middleware handles routing, transformation, and business logic. In microservices, services (endpoints) are smart — they contain their own logic. The pipes (HTTP, Kafka, RabbitMQ) are dumb — they just route messages without understanding them. This keeps services independently deployable and the infrastructure replaceable.

ESB / Smart Pipe (Anti-Pattern)	Microservices / Smart Endpoint (Good)
Routing logic in middleware	Routing logic in service code
Transformation in the bus	Data transformation in receiving service
Orchestration in ESB	Choreography via domain events
Business rules in integration layer	Business rules in domain service
Coupled to bus technology	Infrastructure is pluggable/replaceable

P7 Synchronous vs Asynchronous Communication

Choose the right communication style for each interaction

Aspect	Synchronous (REST/gRPC)	Asynchronous (Kafka/RabbitMQ)
Coupling	Temporal coupling – both services must be up	No temporal coupling – dequeued when ready
Latency	Immediate response	Eventually consistent (higher latency)
Error handling	Immediate failure propagation	Failures handled independently
Use for	Queries, UI-driven requests needing response	Commands, events, fire-and-forget
Examples	GET /product/{id}, POST /orders (returns ID)	OrderPlaced event, PaymentProcessed event
Resilience	Caller fails if callee is down	Messages queued; callee can be down
Complexity	Simple to reason about	More complex (idempotency, ordering)

Choosing sync vs async

```
// Decision guide – when to use what:

// USE SYNCHRONOUS when:
// 1. You need an immediate response to return to the caller
// 2. Query operations (READ data)
// 3. User-facing operations where latency matters

// REST: GET /api/products/123
// gRPC: ProductService.GetProduct(ProductRequest)

// USE ASYNCHRONOUS when:
// 1. The operation is a side effect that can happen later
// 2. Multiple services need to react to the same event
// 3. You want to decouple services temporally
// 4. The operation is slow or unreliable

// Event: OrderPlaced -> PaymentService, InventoryService, NotificationService
// All three react to the same event independently

// HYBRID pattern – command accepts, event confirms:
POST /api/orders // sync: returns orderId + status=PENDING
// OrderPlaced event published asynchronously
// GET /api/orders/{id} // sync: poll for CONFIRMED status
```

P8 Event-Driven Architecture

Services communicate by publishing and consuming domain events

Event-Driven Architecture

In EDA, services communicate by publishing domain events (things that happened) to a message broker. Other services subscribe to events they care about. No service calls another directly; they react to events. This creates maximum decoupling — the producer doesn't know who consumes its events, and consumers don't care who produced them.

Event-Driven – multiple consumers react to one event

```
// Domain Events – immutable facts that something happened
public record OrderPlacedEvent(
String orderId,
String customerId,
List items,
double totalAmount,
Instant occurredAt) {}

// OrderService – publishes event
@Service
public class OrderService {
private final KafkaTemplate kafka;

@Transactional
public Order placeOrder(CreateOrderRequest req) {
Order order = orderRepo.save(new Order(req));
```

```
OrderPlacedEvent event = new OrderPlacedEvent(
order.getId(), req.customerId(), req.items(),
order.getTotal(), Instant.now());
kafka.send("order-placed", order.getId(), event);
return order;
}
}
// PaymentService – reacts to order event
@KafkaListener(topics = "order-placed", groupId = "payment-group")
public void onOrderPlaced(OrderPlacedEvent event) {
paymentService.initiatePayment(event.orderId(), event.totalAmount());
}
// InventoryService – also reacts to same event
@KafkaListener(topics = "order-placed", groupId = "inventory-group")
public void onOrderPlaced(OrderPlacedEvent event) {
inventoryService.reserveItems(event.orderId(), event.items());
}
// NotificationService – also reacts to same event
@KafkaListener(topics = "order-placed", groupId = "notification-group")
public void onOrderPlaced(OrderPlacedEvent event) {
notificationService.sendConfirmation(event.customerId(), event.orderId());
}
// Producer changed nothing; it doesn't know any of these consumers exist
```

P9 Consumer-Driven Contract Testing

API changes are tested against real consumer expectations

Consumer-Driven Contract Testing

In microservices, API changes in a provider can silently break consumers. Consumer-Driven Contracts (CDC) solve this: each consumer defines the contract it expects from the provider (via Pact tests), and the provider's CI pipeline verifies that it still satisfies all registered consumer contracts before deploying. This gives confidence for independent deployments.

Pact – Consumer-Driven Contract test

```
// Consumer (OrderService) writes a Pact test
// "I expect ProductService to return this response for GET /products/1"
@Pact(consumer = "OrderService", provider = "ProductService")
public RequestResponsePact pact(PactDslWithProvider builder) {
return builder
.given("Product 1 exists")
.uponReceiving("GET product by id")
.path("/api/products/1")
.method("GET")
.willRespondWith()
.status(200)
```

```
    .body(new PactDslJsonBody()
    .integerType("id", 1)
    .stringType("name", "Laptop")
    .numberType("price", 999.99)
    .booleanType("inStock", true))
    .toPact();
    }
    // ProductService provider verifies: does my API satisfy this contract?
    // @PactVerification("OrderService")
    // Runs in ProductService CI/CD pipeline before deploy
    // If the contract is not met -> pipeline FAILS -> cannot deploy
```

CHAPTER 4

Resilience & Fault Tolerance Principles

Design for Failure, Bulkhead Isolation & Circuit Breaker

P10 Design for Failure

Assume every network call, dependency, and service WILL fail

Design for Failure

In a distributed system, network partitions, timeouts, slow responses, and service crashes are not exceptional events — they are normal. Every microservice must be designed assuming its dependencies will fail. This means: always set timeouts, implement retries with exponential backoff, provide graceful degradation and fallbacks, and never let one failing downstream service cascade into a full system failure.

- **Timeout** – Never make an unbounded network call. Set connection AND read timeouts on every outbound call.
- **Retry with backoff** – Retry transient failures with exponential backoff + jitter to avoid thundering herd.
- **Fallback** – Always provide a meaningful fallback when a dependency fails (cached data, default, empty list).
- **Graceful degradation** – Partial failure should degrade functionality, not crash the whole service.
- **Idempotency** – Retried operations must be safe to execute multiple times without side effects.
- **Fail fast** – Detect failure quickly and short-circuit; don't let failures queue up.

Resilience4j – circuit breaker + retry + timeout + fallback

```
// GOOD: Design for Failure with Resilience4j
@Service
public class ProductClient {
private final RestTemplate restTemplate;

@CircuitBreaker(name = "productService",
fallbackMethod = "productFallback")
@Retry(name = "productService")
@TimeLimiter(name = "productService")
public CompletableFuture getProduct(String id) {
return CompletableFuture.supplyAsync(() ->
restTemplate.getForObject(
"http://product-service/api/products/" + id,
ProductDTO.class));
}

// Fallback: return cached/default data when service is unavailable
public CompletableFuture productFallback(
String id, Throwable t) {
```

```
log.warn("ProductService unavailable for {}: {}", id, t.getMessage());
return CompletableFuture.completedFuture(
ProductDTO.builder()
.id(id)
.name("Product Unavailable")
.price(0.0)
.available(false)
.build());
}
}
# application.yml – configure timeout, retry, circuit breaker
resilience4j:
timelimiter.instances.productService.timeout-duration: 3s
retry.instances.productService:
max-attempts: 3
wait-duration: 500ms
retry-exceptions: [java.net.ConnectException]
circuitbreaker.instances.productService:
failure-rate-threshold: 50
wait-duration-in-open-state: 10s
sliding-window-size: 10
```

P11 Bulkhead & Isolation

Isolate failures to prevent one failing dependency from consuming all resources

Bulkhead Pattern

Named after the compartments in a ship hull that prevent it from sinking when one section floods, the Bulkhead pattern limits the resources (threads, connections) allocated to each downstream service. If PaymentService becomes slow and consumes all thread-pool resources, it should NOT affect OrderService's ability to call ProductService. Each dependency gets its own isolated resource pool.

Bulkhead thread pool isolation

```
# Bulkhead: separate thread pools per service dependency
resilience4j:
thread-pool-bulkhead:
instances:
paymentService:
max-thread-pool-size: 5 # max 5 threads for payment calls
core-thread-pool-size: 3
queue-capacity: 10
productService:
max-thread-pool-size: 10 # separate pool for product calls
core-thread-pool-size: 5
queue-capacity: 20
// If PaymentService is slow and its 5 threads + 10 queue slots fill up,
```

```
// calls to ProductService on their separate pool are NOT affected.
// Payment calls fast-fail; product calls continue normally.
```

P12 Circuit Breaker & Fallback

Stop calling a failing service; fail fast and recover gracefully

Circuit State	Behaviour	Condition to Change
CLOSED	Normal: calls pass through, failures counted	Failure rate exceeds threshold -> OPEN
OPEN	Fast-fail: calls immediately return fallback	After wait-duration -> HALF-OPEN
HALF-OPEN	Test: N probe calls allowed through	Probes succeed -> CLOSED; fail -> OPEN

CHAPTER 5

Data Management Principles

Database Per Service, Eventual Consistency, Saga, CQRS & Event Sourcing

P13 Database Per Service

Each service owns its own data store with no shared database access

Database Per Service

This is the most critical — and most frequently violated — microservices principle. Each service must own its data and be the sole source of truth for that data. No other service may directly query another service's database. If Service B needs data owned by Service A, it must call Service A's API or subscribe to Service A's events. Shared databases create tight coupling that negates the benefits of microservices.

+ Good Practice

```
# GOOD: Database per service

OrderService -> PostgreSQL (orders, order_items)
ProductService -> PostgreSQL (products, categories)
UserService -> PostgreSQL (users, addresses)
SessionService -> Redis (sessions, tokens)
SearchService -> Elasticsearch (product index)
AnalyticsService -> ClickHouse (event data)

# Services communicate via API calls or events
# Each team chooses the best DB for their use case
```

- Anti-Pattern

```
# BAD: Shared database

OrderService --|
ProductService --|-> Shared PostgreSQL
UserService --|
PaymentService --|

# Problems:
# - Schema change in one service breaks others
# - Services cannot be deployed independently
# - Single DB = single point of failure
# - Cannot choose optimal DB per service
# - All teams wait for schema migration sign-off
```

P14 Eventual Consistency & Saga Pattern

Distributed transactions managed as compensatable local transactions

Saga Pattern

Without a shared database, you cannot use a single ACID transaction across services. The Saga pattern manages distributed transactions as a sequence of local transactions, each publishing an event to trigger the next step. If a step fails, compensating transactions are executed to undo the previous steps. The system reaches eventual consistency, not immediate consistency.

Saga choreography – order creation flow

```
// Choreography-based Saga (event-driven)
// No central coordinator; services react to each other's events
```

```
// Step 1: OrderService creates PENDING order, publishes event
OrderService --[OrderPlaced event]--> Kafka

// Step 2: InventoryService reserves stock
InventoryService <-- [OrderPlaced]
-> reserves stock
-> publishes [StockReserved] OR [StockInsufficient]

// Step 3: PaymentService charges
PaymentService <-- [StockReserved]
-> charges card
-> publishes [PaymentSucceeded] OR [PaymentFailed]

// Step 4: OrderService confirms
OrderService <-- [PaymentSucceeded]
-> updates order to CONFIRMED
-> publishes [OrderConfirmed]

// Compensation (rollback path):
OrderService <-- [PaymentFailed]
-> publishes [ReleaseStockRequest]
InventoryService <-- [ReleaseStockRequest]
-> releases reserved stock
OrderService -> marks order CANCELLED
```

P15 CQRS & Event Sourcing

Separate reads from writes for scalability and audit history

CQRS (Command Query Responsibility Segregation)

CQRS separates the write model (Commands: create, update, delete) from the read model (Queries: fetch data). The write side uses a normalised database optimised for consistency. The read side uses a denormalised, query-optimised store (Redis, Elasticsearch, NoSQL) updated by listening to domain events. This enables each side to scale independently.

CQRS – separate command and query controllers with projection

```
// Command side – handles writes
@RestController @RequestMapping("/api/orders")
public class OrderCommandController {
@PostMapping
public ResponseEntity create(@RequestBody CreateOrderCommand cmd) {
String orderId = orderCommandService.handle(cmd);
return ResponseEntity.accepted().body(orderId);
}
}

// Query side – serves reads from denormalised view
@RestController @RequestMapping("/api/orders/query")
public class OrderQueryController {
private final OrderViewRepository viewRepo;
```

```
@GetMapping("/customer/{customerId}")
public List getOrders(@PathVariable String customerId) {
return viewRepo.findByCustomerId(customerId);
}
}
// Projection: builds read model from events
@EventHandler
public void on(OrderPlacedEvent event) {
viewRepo.save(new OrderView(event.orderId(), event.customerId(),
"PENDING", event.totalAmount()));
}
@EventHandler
public void on(OrderConfirmedEvent event) {
viewRepo.updateStatus(event.orderId(), "CONFIRMED");
}
```

CHAPTER 6

Observability Principles

Centralised Logging, Distributed Tracing, Metrics & The Three Pillars

6.1 The Three Pillars of Observability

Pillar	What It Answers	Tools	Key Data
Logs	What happened?	ELK, Loki, Splunk	Structured JSON logs with traceId, service, level
Metrics	How is it performing?	Prometheus, Grafana	Request rate, latency, error rate, saturation
Traces	Why did it happen?	Zipkin, Jaeger, Tempo	Full request path across all services + durations

Three pillars implementation with Spring Boot

```
# Every microservice MUST implement all three pillars

# 1. STRUCTURED LOGS – JSON with mandatory fields
{
"@timestamp": "2025-01-15T10:30:00Z",
"level": "INFO",
"service": "order-service",
"traceId": "abc123def456", // same across all services
"spanId": "1234abcd",
"message": "Order created",
"orderId": "ORD-789",
"customerId": "USR-123",
"durationMs": 42
}

# 2. METRICS – expose /actuator/prometheus
# Key metrics every service should expose:
# http_server_requests_seconds (latency P50, P95, P99)
# http_server_requests_total (request rate)
# jvm_memory_used_bytes
# db_connection_pool_active
# kafka_consumer_lag_count (for consumers)

# 3. TRACES – auto-propagate traceId
# With Micrometer + Zipkin, traceId is automatically
# propagated via HTTP headers (X-B3-TraceId) across
# all service calls in a request chain
```

```
# Alert thresholds (Prometheus AlertManager rules):
# - Error rate > 1% for 5 minutes
# - P99 latency > 2 seconds
# - Service unavailable (no /health response)
# - Consumer lag > 10,000 messages
```

CHAPTER 7

Security Principles

Zero Trust, Auth, Secrets Management & Network Security

7.1 Zero Trust Security Model

Zero Trust

Zero Trust means 'never trust, always verify'. In a microservices architecture, no service is implicitly trusted just because it is inside the network perimeter. Every service-to-service call must be authenticated (mTLS certificates or JWT tokens) and authorised (the caller has the right to make this specific request).

Security Layer	Implementation	Tool
Authentication	Every service call carries a JWT token or mTLS cert	Spring Security, Istio, JWT
Authorisation	JWT scopes/roles checked at each service	@PreAuthorize, RBAC
Network Isolation	NetworkPolicy restricts which pods can talk to which	Kubernetes NetworkPolicy
mTLS	Mutual TLS encrypts and authenticates all service calls	Istio Service Mesh
Secrets	Passwords/keys never in config files or environment	HashiCorp Vault, K8s Secrets
API Gateway Auth	Validate JWT once at edge; propagate claims inward	Kong, Spring Cloud Gateway
Input Validation	Validate all inputs at service boundary	@Valid, Bean Validation

JWT propagation and service-level authorisation

```
// JWT propagation across services
// API Gateway validates token, extracts claims, passes to services

// Feign interceptor – propagate JWT to downstream calls
@Component
public class JwtFeignInterceptor implements RequestInterceptor {
@Override
public void apply(RequestTemplate template) {
// Extract token from current security context
Authentication auth = SecurityContextHolder
.getContext().getAuthentication();
if (auth instanceof JwtAuthenticationToken jwt) {
template.header("Authorization",
```

```
"Bearer " + jwt.getToken().getTokenValue());
}
}
}
// Service-level authorisation
@RestController
@RequestMapping("/api/admin")
public class AdminController {
@GetMapping("/users")
@PreAuthorize("hasRole('ADMIN') or hasAuthority('SCOPE_admin:read')")
public List getAllUsers() { ... }
}
```

CHAPTER 8

Deployment & Operational Principles

Twelve-Factor App, Immutable Infrastructure & Independent Deployability

8.1 Twelve-Factor App Principles

Factor	Principle	Implementation
I. Codebase	One codebase, many deploys	One git repo per service
II. Dependencies	Explicitly declare dependencies	pom.xml / package.json with pinned versions
III. Config	Config in environment, not code	application.yml + env vars; no hardcoded values
IV. Backing services	Treat as attached resources	Connect via URL/config; swap DB without code change
V. Build/Release/Run	Strict stages separation	Build once, deploy many environments
VI. Processes	Execute as stateless processes	No session in memory; use Redis/DB for state
VII. Port Binding	Export via port binding	Embedded Tomcat; service listens on a port
VIII. Concurrency	Scale via process model	Horizontal scaling; no reliance on single process
IX. Disposability	Fast startup, graceful shutdown	SIGTERM handling; short startup time
X. Dev/Prod Parity	Keep envs as similar as possible	Docker; same image in dev and prod
XI. Logs	Treat logs as event streams	Log to stdout; let infrastructure handle routing
XII. Admin Processes	Run admin as one-off processes	Migrations as init containers or Jobs

8.2 Independent Deployability

Independent Deployability

Every microservice should be deployable at any time without coordinating with other teams. This is the ultimate test of whether your services are truly microservices. To achieve it: maintain backward-compatible API changes, use API versioning, tolerate old and new versions running simultaneously (expand-contract pattern), and never require simultaneous deployment of multiple services.

Expand-contract and API versioning for independent deployability

```
# Expand-Contract pattern for zero-downtime database changes

# Phase 1 EXPAND: Add new column (nullable) – deploy service v1.1
ALTER TABLE orders ADD COLUMN reference_code VARCHAR(20);
# Both old v1.0 and new v1.1 work – old ignores new column

# Phase 2 MIGRATE: Backfill existing rows – background job
```

```
UPDATE orders SET reference_code = 'ORD-' || id WHERE reference_code IS NULL;

# Phase 3 CONTRACT: Make column mandatory – deploy service v1.2
ALTER TABLE orders ALTER COLUMN reference_code SET NOT NULL;
# Only after ALL instances of v1.0 are replaced with v1.2

# API Versioning – never break existing consumers
/api/v1/orders <- old endpoint, still works
/api/v2/orders <- new endpoint with changed response shape
# Run both until all consumers are migrated to v2
```

CHAPTER 9

Team & Organisational Principles

Conway's Law, Team Topologies & Decentralised Governance

9.1 Conway's Law

Conway's Law

"Organisations which design systems are constrained to produce designs which are copies of the communication structures of those organisations." — Mel Conway, 1967. In practice, if four teams are working on a compiler, you'll get a four-pass compiler. Microservices must be aligned to team boundaries. Each team fully owns its service(s) — from development through production. Services owned by many teams become the hardest to change.

Conway's Law Implication	Microservices Consequence
Teams communicate via meetings	Services communicate via APIs – meeting = API coupling
Large team, shared codebase	Monolith: many hands in the same pot
Small autonomous team	One or few microservices; clear ownership
Two teams own one service	Service has conflicting priorities; hard to release
Org restructured	Services should be restructured to match

9.2 You Build It, You Run It

You Build It, You Run It

This principle, popularised by Amazon's Werner Vogels, means the team that develops a microservice is also responsible for operating it in production — on-call, monitoring, incident response. This creates a tight feedback loop: developers feel the pain of their operational decisions, leading to better-designed systems, better logging, better runbooks, and a culture of operational excellence.

- Team owns the full lifecycle: design → build → test → deploy → monitor → incident.
- No 'throwing over the wall' to Ops — DevOps is a culture, not a team.
- Team maintains their own monitoring dashboards, alerts, and runbooks.
- On-call rotation is within the team, not a central Ops team.
- Postmortems are owned by the team to drive systemic improvement.

9.3 Decentralised Governance & Team Topology

Team Type (Team Topologies)	Role	Interaction
Stream-Aligned Team	Owns and delivers a business-domain service end-to-end	Primary delivery team
Platform Team	Provides internal platform (CI/CD, K8s, monitoring) as a service	Enablement; reduces cognitive load
Enabling Team	Helps stream teams adopt new practices/tools	Temporary; builds capability
Complicated-Subsystem Team	Owns complex technical areas (ML, security, media)	Expert support

CHAPTER 10

Anti-Patterns, Common Mistakes & Checklist

What to Avoid, Red Flags & The Complete Design Principles Checklist

10.1 The Top Microservices Anti-Patterns

Anti-Pattern	What It Is	Why It's Dangerous	How to Fix
Distributed Monolith	Services that are deployed separately but cannot run independently; all must be deployed at once	You get all the complexity of microservices with none of the benefits	Remove shared databases, fix tight coupling, enable independent deployability
Chatty Services	Service A calls B, which calls C, which calls D synchronously for one request	Cascading failures; high latency; any one failure breaks the chain	Use async events; BFF pattern to aggregate; batch APIs
Shared Database	Multiple services read/write the same database tables	Schema changes require coordinating all services; no independent deployability	Database per service; expose data via APIs or events
Synchronous Call Chains	Order -> Payment -> Inventory -> Notification all in one sync chain	Temporal coupling; if Notification is slow, entire order flow is slow	Use async messaging for non-blocking operations
Anemic Services	Services with no business logic; just CRUD wrappers over a shared DB	Signals services are too fine-grained or incorrectly bounded	Align to Bounded Contexts; add domain logic to services
Missing Observability	Services with no structured logging, metrics, or tracing	Impossible to diagnose production issues across service boundaries	Implement all three pillars: logs, metrics, traces
Versioning Ignored	API changes break consumers; no versioning strategy	Services cannot be deployed independently; consumers break silently	Consumer-driven contracts; API versioning; backward-compatible changes
No Circuit Breaker	Calling services with no timeout/retry/fallback	One slow dependency degrades entire system; cascading failures	Resilience4j: CircuitBreaker + Retry + TimeLimiter + Fallback

10.2 Microservices Design Principles – Complete Checklist

Principle	Checklist Questions	Status
Single Responsibility	Does each service have exactly ONE business reason to change?	[ ]

Principle	Checklist Questions	Status
Bounded Context	Are service boundaries aligned to business domain boundaries?	[ ]
Loose Coupling	Can each service be deployed without requiring changes in others?	[ ]
High Cohesion	Does all functionality in the service belong together?	[ ]
API-First	Was the API contract defined before implementation began?	[ ]
Smart Endpoints	Does business logic live in services, not the message bus?	[ ]
Async Communication	Are fire-and-forget operations using messaging, not sync calls?	[ ]
Event-Driven	Are domain events published for cross-service state changes?	[ ]
Consumer Contracts	Do consumer Pact tests run in provider's CI/CD pipeline?	[ ]
Design for Failure	Does every outbound call have timeout + retry + fallback?	[ ]
Bulkhead	Are thread pools isolated per downstream dependency?	[ ]
Circuit Breaker	Is Resilience4j or equivalent configured on all outbound calls?	[ ]
Database Per Service	Does NO service directly access another service's database?	[ ]
Eventual Consistency	Are distributed transactions handled via Saga pattern?	[ ]
CQRS	Are read models separated from write models for complex queries?	[ ]
Structured Logging	Does every log entry include traceId, service name, level?	[ ]
Distributed Tracing	Does the traceId propagate across all service calls?	[ ]
Metrics	Does every service expose /metrics with business + technical KPIs?	[ ]
Zero Trust	Is every service-to-service call authenticated and authorised?	[ ]
Secrets Management	Are passwords/tokens stored in Vault/K8s Secrets, not config files?	[ ]
Independent Deploy	Can each service be deployed independently at any time?	[ ]
Stateless Processes	Does the service store session state externally (Redis, DB)?	[ ]
Config Externalised	Is ALL environment-specific config external (not in the image)?	[ ]
Graceful Shutdown	Does the service handle SIGTERM and finish in-flight requests?	[ ]
Team Ownership	Does exactly one team own each service end-to-end?	[ ]

+ Start simple. Not every principle applies from day one. Begin with Single Responsibility, Loose Coupling, Database Per Service, and Design for Failure. Add Event-Driven, CQRS, and Consumer Contracts as the system matures.

! The most common failure mode is the Distributed Monolith — services that are separately deployed but share a database and cannot change independently. If you can't answer 'yes' to independent deployability, you may have a monolith distributed across the network, which is worse than a single monolith.